CELL 6 Split Guide - Resumable Cells

Overview

CELL 6 has been split into **3 resumable cells** for better troubleshooting and flexibility:

- **CELL 6A**: Basic cumulative metrics (always required, fast ~30 min)
- **CELL 6B**: Optional metrics (ratings given, grade, last4) - can skip
- **CELL 6C**: Pool metrics (optional, slowest ~1-2 hours) - can skip

Benefits

1. **Resumable**: Each cell checks for its output file and skips computation if found
2. **Flexible**: Can skip expensive parts (6B, 6C) if not needed
3. **Troubleshooting**: If one cell crashes, you still have previous cell's output
4. **Progress tracking**: Clear checkpoints at each stage
5. **Automatic cleanup**: Ray objects and temp directory cleaned between cells to prevent memory issues

Cell Structure

CELL 6A: Basic Metrics (Always Required)

Input: `df_pipeline_02_oer_joined` (from CELL 5) **Output**: `df_pipeline_03a_basic_metrics`

Creates: - `cum_tb_rcvd_rtr`, `cum_tb_rcvd_snr` - `cum_evals_rcvd_rtr`, `cum_evals_rcvd_snr` - `tb_ratio_rtr`, `tb_ratio_snr` (key columns for Cox analysis)

Time: ~30 minutes

Resume: If `df_pipeline_03a_basic_metrics.feather` exists, cell skips computation

Cleanup: Automatically runs `ray_cleanup()` and cleans Ray temp directory after completion (before CELL 6B)

CELL 6B: Optional Metrics (Can Skip)

Input: `df_pipeline_03a_basic_metrics` (from CELL 6A) **Output**: `df_pipeline_03b_optional_metrics`

Creates (if enabled in `CELL6_OPTIMIZE`): - Ratings given: `cum_tb_given_rtr`, `cum_tb_given_snr`, etc. - Grade metrics: `co_grade_tb_rtr`, `field_grade_tb_rtr`, etc. - Last4 metrics: `last4_tb_rtr`, `last4_tb_snr`, etc.

Time: ~30-60 minutes total (if all enabled)

Resume: If `df_pipeline_03b_optional_metrics.feather` exists, cell skips computation

Cleanup: Automatically runs `ray_cleanup()` and cleans Ray temp directory after completion (before CELL 6C)

Skip: Set `CELL6B = False` or disable in `CELL6_OPTIMIZE`

CELL 6C: Pool Metrics (Can Skip - Slowest)

Input: `df_pipeline_03b_optional_metrics` (from CELL 6B) or `df_pipeline_03a_basic_metrics` (from CELL 6A) **Output**: `df_pipeline_03_oer_metrics` (final output)

Creates (if enabled in `CELL6_OPTIMIZE`): - Pool means, differences, ranks (28 columns) - Comparative metrics within rating pools

Time: ~1-2 hours (slowest part)

Resume: If `df_pipeline_03_oer_metrics.feather` exists, cell skips computation

Skip: Set `CELL6C = False` or `CELL6_OPTIMIZE['include_pool_metrics'] = False`

Automatic Memory Cleanup

To prevent memory issues and temp directory overflow, **automatic cleanup runs between cells**:

- **After CELL 6A**: Runs `ray_cleanup()` and cleans Ray temp directory before starting CELL 6B
- **After CELL 6B**: Runs `ray_cleanup()` and cleans Ray temp directory before starting CELL 6C

This helps prevent: - Memory buildup from Ray object references - Temp directory overflow (which can cause crashes) - Object store exhaustion

Note: Cleanup only runs after successful computation. If a cell is skipped (output already exists), cleanup is not needed.

How to Use

Option 1: Run All Cells (Full Pipeline)

```
# In pipeline_config.py:
CELL6A = True
CELL6B = True
CELL6C = True
# Or use legacy flag:
CELL6 = True  # Runs all 6A, 6B, 6C
```

Total time: ~2-3 hours

Option 2: Skip Optional Metrics (Fastest for Basic Analysis)

```
# In pipeline_config.py:
CELL6A = True   # Required
CELL6B = False  # Skip optional metrics
CELL6C = False  # Skip pool metrics
```

Total time: ~30 minutes (just basic metrics + TB ratios)

Option 3: Skip Pool Metrics Only

```
# In pipeline_config.py:
CELL6A = True
CELL6B = True
CELL6C = False  # Skip pool metrics (saves 1-2 hours)
```

Total time: ~1 hour (basic + optional, no pool)

Option 4: Resume from Checkpoint

If a cell crashes or is interrupted:

1. **Check which files exist**:
   - `df_pipeline_03a_basic_metrics.feather` - CELL 6A complete
   - `df_pipeline_03b_optional_metrics.feather` - CELL 6B complete
   - `df_pipeline_03_oer_metrics.feather` - CELL 6C complete
2. **Resume from last checkpoint**:
   - If 6A exists but 6B doesn't: Run CELL 6B (it will load from 6A)
   - If 6B exists but 6C doesn't: Run CELL 6C (it will load from 6B)
   - Each cell automatically checks for its input and output files
3. **Example**: If CELL 6C crashes after 1 hour:

   ```
   # Just re-run CELL 6C - it will detect that 6B output exists
   CELL6C = True
   # Cell will load df_pipeline_03b_optional_metrics and continue
   ```

File Dependencies

```
CELL 5 Output
    ↓
df_pipeline_02_oer_joined.feather
    ↓
CELL 6A (always required)
    ↓
df_pipeline_03a_basic_metrics.feather
    ↓
CELL 6B (optional)
    ↓
df_pipeline_03b_optional_metrics.feather
    ↓
CELL 6C (optional, slowest)
    ↓
df_pipeline_03_oer_metrics.feather (final output)
```

Troubleshooting

If CELL 6A fails:

- Check that CELL 5 completed successfully
- Verify `df_pipeline_02_oer_joined.feather` exists

If CELL 6B fails:

- Check that CELL 6A completed successfully
- Verify `df_pipeline_03a_basic_metrics.feather` exists
- Can skip CELL 6B if optional metrics not needed

If CELL 6C fails:

- Check that CELL 6A (or 6B) completed successfully
- Can skip CELL 6C if pool metrics not needed
- If needed, can resume by re-running CELL 6C (it will load from previous checkpoint)

To start fresh:

Delete the checkpoint files: - `df_pipeline_03a_basic_metrics.feather` - `df_pipeline_03b_optional_metrics.feather` - `df_pipeline_03_oer_metrics.feather`

Then re-run from CELL 6A.

Performance Notes

Important Limitation: The `add_cumulative_oer_metrics()` function currently recomputes all metrics each time it's called. This means: - CELL 6B will recompute basic metrics (even though they exist from 6A) - ~30 min redundant work - CELL 6C will recompute basic + optional metrics (even though they exist from 6B) - ~1 hour redundant work

However, this split is still **very useful** because: 1. ✅ You can **skip** 6B and 6C entirely if not needed (saves 1-2 hours) 2. ✅ Each cell **saves its output**, so you can resume from any checkpoint 3. ✅ If 6C crashes, you still have 6B output (with optional metrics) 4. ✅ Better troubleshooting - can test each part independently 5. ✅ Clear progress tracking - know exactly which stage completed

Best Practice: - If you only need basic metrics: Run CELL 6A only (fastest, ~30 min) - If you need optional metrics: Run 6A + 6B (accepts ~30 min redundancy, ~1 hour total) - If you need everything: Run all three (accepts redundancy, but can resume from checkpoints)

Future Improvement: The function could be enhanced to accept "add only X metrics" mode to eliminate redundancy.

Recommended Workflow

1. **First run**: Run all cells to get complete metrics
2. **Subsequent runs**:
    - If only basic metrics needed: Run CELL 6A only

- If pool metrics crash: Re-run CELL 6C (it will resume from 6B)

3. **Production**: Disable optional features you don't need to save time

Configuration

Control which metrics are computed in `CELL6_OPTIMIZE` (`pipeline_config.py`):

```
CELL6_OPTIMIZE = {
    'include_ratings_given': True,   # Used in CELL 6B
    'include_grade_metrics': True,   # Used in CELL 6B
    'include_last4_metrics': True,   # Used in CELL 6B
    'include_pool_metrics': True,    # Used in CELL 6C
    'use_optimized_pool_metrics': True,  # Optimized pool metrics path
    'check_data_quality': False,     # Set False if data already clean
    'profile_performance': True,     # Enable profiling
}
```

Legacy Support

The old `CELL6` flag still works - if `CELL6 = True`, it runs all three cells (6A, 6B, 6C) in sequence.